Filière de Big Data et Intelligence Artificielle
École Nationale des Sciences Appliquées de Tétouan

Data Mining Report

Analysis and prediction of AI Tool Usage by College Students and it's impact



Presented by:
SOULAMA DOUBIE JUDICAEL Noé
EL ALAOUI SALEH Zouhair
PROBI Nouhaila
SIMBRE Achraf

Supervised by:
Pr.BELCAID Anass

2025-2026

Appreciation

This project represents a meaningful academic experience that required dedication, discipline, and intellectual effort.
We would like to sincerely and deeply express our appreciation to Mr. Belcaid Anass, whose guidance, availability, and academic rigor were consistently essential to the successful completion of this work. His insightful remarks and methodological support significantly contributed to the depth and overall quality of this project.

Contents

1 Introduction **6**
- 1.1 Project Objective . . . 6
- 1.2 Dataset Description . . . 6
- 1.3 General Methodology . . . 7
- 1.4 Significance of the Study . . . 8

2 Exploratory Data Analysis **9**
- 2.1 Importing required libraries . . . 9
- 2.2 Data loading and Dataset overview . . . 10
  - 2.2.1 Data loading from kaggle . . . 10
  - 2.2.2 Data overview . . . 11
- 2.3 Data Cleaning . . . 12
  - 2.3.1 Handling Missing Values . . . 12
  - 2.3.2 Removing Duplicate Records . . . 14
- 2.4 Univariate Analysis . . . 14
  - 2.4.1 Distribution of the Target Variable . . . 14
  - 2.4.2 Distribution of Numerical Features (Histograms) . . . 15
  - 2.4.3 Distribution of Categorical Features . . . 16
  - 2.4.4 Outlier Detection . . . 17
- 2.5 Multivariate Analysis . . . 17
  - 2.5.1 Relationship Between Numerical Features(Scatter/Line Plots) . . . 17
  - 2.5.2 Analysis of Features vs Target Variable . . . 18
  - 2.5.3 Correlation Matrix and Heatmap . . . 19

3 Data Preparation for Modeling **20**
- 3.1 Encoding Strategies and Technical Justifications . . . 20
  - 3.1.1 Multi-Label Binarization (MLB) – Handling Multi-choice data . . . 20
  - 3.1.2 One-Hot Encoding (OHE) – Maintaining Neutrality . . . 22
  - 3.1.3 Ordinal Encoding and Preserving Hierarchical Integrity . . . 23
- 3.2 Data Scaling . . . 24
- 3.3 Feature Selection and Correlation Analysis . . . 25
- 3.4 Validation via P-Value: Scientific Proof . . . 27
- 3.5 Final Optimization . . . 31
- 3.6 Data Splitting (Training and Testing Sets) . . . 31
  - 3.6.1 Reasons for Splitting the Data . . . 31
  - 3.6.2 Data Splitting Method . . . 32

4 Classification Models **33**
- 4.1 Multi-class Classification . . . 33
  - 4.1.1 DecisionTreeClassifier . . . 33
  - 4.1.2 Model Implementation . . . 33

4.1.3 Results and Confusion Matrix 33
4.2 Binary classification 34
4.2.1 Definition of each Model 34
4.2.2 Model Implementation 35
4.2.3 Results and Visualisation 35
4.2.4 Confusion Matrix 37

5 Clustering **39**
5.1 Feature Selection 39
5.2 The Optimal Number of Clusters (Elbow Method) 39
5.3 Training the K-Means Model 40
5.4 Visualizing Clusters in 2D (PCA) 41
5.5 Cluster Profiling (Analysis) 42

6 Conclusion **44**
6.1 Project Workflow Summary 44
6.2 Key Findings 44
6.3 Study Limitations and Contributions 45
6.4 Future Work and Perspectives 45

List of Figures

2.1 Missing data visualization 13
2.2 Missing data visualization after cleaning 14
2.3 density distribution of the target variable. 15
2.4 Density of daily usage 16
2.5 AI tools usage 16
2.6 impact of access to internet on daily usage 18
2.7 Impact of using AI on grades 18
2.8 Correlation Matrix of Selected Numerical Variables 19

3.1 Boxplots of Key Numerical Variables Before Scaling 24
3.2 Correlation of Encoded Features with Academic Impact 26
3.3 Heatmap of Correlations Among Selected Significant Features 27
3.4 Joint Distribution of Daily AI Usage Hours and Academic Impact 29
3.5 Frequency Distribution of AI Use Cases Among Students 30
3.6 Comparison of Daily AI Usage Based on Professors' Permission 31

4.1 Confusion Matrix of the Multi-class Decision Tree Classifier 34
4.2 Performance Comparison of Binary Classification Models 37
4.3 Confusion Matrices of Binary Classification Models 38

5.1 Elbow Method for Determining the Optimal Number of Clusters 40
5.2 Two-Dimensional Visualization of Student Clusters Using PCA 41
5.3 Cluster Profiling Based on Mean Encoded Feature Values 43

List of Tables

2.1 Dataset structure and data types 11
2.2 Statistical summary of numerical variables 12
2.3 Number of missing values per feature 12

3.1 Use Cases Binary Encoded 21
3.2 AI Tools Usage Binary Encoded 22
3.3 Internet Access Encoding 23
3.4 Binary Encoding of Professors' Permission and Willingness to Pay 23
3.5 Statistically Significant Correlations 28

5.1 Clustering Based on Daily Usage Hours 40

Chapter 1

Introduction

1.1 Project Objective

The main objective of this project is to analyze and model the usage of Artificial Intelligence (AI) tools by college students and to evaluate their impact on academic performance.

More specifically, this study aims to:

- Explore students' patterns of AI tool usage, including frequency, preferred tools, and use cases.
- Identify key factors influencing daily AI usage such as internet access, awareness level, and institutional acceptance.
- Assess the relationship between AI usage behavior and its perceived impact on students' grades.
- Apply machine learning techniques to predict academic impact based on students' AI usage profiles.
- Segment students into meaningful groups using clustering techniques to better understand behavioral differences.

Through this data-driven approach, the project seeks to provide actionable insights into how AI tools are reshaping learning practices in higher education.

1.2 Dataset Description

The dataset used in this project is sourced from Kaggle and focuses on the usage of Artificial Intelligence tools by Indian college students in 2025. It is publicly available at the following link: **https://www.kaggle.com/datasets/rakeshkapilavai/ai-tool-usage-by-indian-college-students-2025/data**

The dataset contains 3,614 observations, each representing an individual student, and includes a diverse set of variables describing academic background, AI usage behavior, perceptions toward AI technologies, and contextual conditions. It provides a rich foundation for exploratory data analysis, supervised learning, and clustering.

The variables included in the dataset are defined as follows:

- **Student Name:** Identifier of the student
- **College Name:** Name of the academic institution attended by the student.

- **Stream:** Academic discipline or field of study (e.g., Engineering, Science, Arts).
- **Year of Study:** Current academic year of the student.
- **AI Tools Used:** List of AI tools used by the student; multiple selections are possible.
- **Daily Usage Hours:** Average number of hours per day spent using AI tools.
- **Use Cases:** Academic purposes for which AI tools are used.
- **Impact on Grades:** Student's perceived academic impact of AI tool usage, ranging from negative to positive values. This variable represents the target variable of the study.
- **Do Professors Allow Use:** Indicates whether professors permit the use of AI tools for academic activities.
- **Preferred AI Tool:** The AI tool most frequently used by the student.
- **Awareness Level:** Level of awareness and familiarity with AI technologies.
- **Willing to Pay for Access:** Indicates whether the student is willing to pay for access to AI tools.
- **State:** Geographic state of the student.
- **Device Used:** Type of device used to access AI tools.
- **Internet Access:** Quality of internet connectivity, categorized as Poor, Medium, or High.

Overall, this dataset offers a comprehensive view of students' interactions with AI tools and enables an in-depth analysis of usage patterns and their perceived academic impact, which aligns directly with the objectives of this project.

1.3 General Methodology

The methodology adopted in this project follows a structured data mining pipeline to ensure analytical rigor and reproducibility.

First, an exploratory data analysis (EDA) is conducted to understand the dataset structure, distributions, and relationships between variables. This step includes data visualization, univariate and multivariate analyses, and detection of anomalies.

Next, extensive data preprocessing is performed, including handling missing values, removing duplicates, encoding categorical variables, scaling numerical features, and selecting the most relevant attributes.

Next, supervised learning techniques are applied to perform both multi-class and binary classification tasks in order to predict the academic impact of AI usage. Model performance is evaluated using appropriate metrics such as accuracy, F1-score, recall, and confusion matrices.

Finally, unsupervised learning through clustering is employed to segment students into distinct behavioral groups, providing deeper insights into AI usage patterns and learning behaviors.

1.4 Significance of the Study

The growing adoption of AI tools in education raises important questions regarding their effectiveness, ethical use, and long-term impact on learning outcomes. This study contributes to this discussion by providing empirical evidence based on real student data.

From an academic perspective, the findings help educators and institutions better understand how students interact with AI technologies and how these tools influence academic performance.

From a practical standpoint, the results can support decision-making related to AI integration in educational policies, such as defining guidelines for acceptable use, improving digital literacy, and identifying students who may benefit the most from AI-assisted learning.

Moreover, this project demonstrates the application of data mining and machine learning techniques to real-world educational data, highlighting their value in uncovering hidden patterns and supporting evidence-based conclusions.

Chapter 2

Exploratory Data Analysis

2.1 Importing required libraries

In this section, we import the required libraries and modules into the notebook. They provide essential functionalities for data analysis and modeling, such as data manipulation, visualization, and machine learning algorithms.

```
import kagglehub
import os

# Core libraries
import numpy as np
import pandas as pd

# Visualization
import seaborn as sns


import matplotlib.pyplot as plt
# Preprocessing
from sklearn.preprocessing import (
    MultiLabelBinarizer,
    OrdinalEncoder,
    LabelBinarizer,
    OneHotEncoder,
    StandardScaler,
    MinMaxScaler
)

# Model selection
from sklearn.model_selection import train_test_split

# Regression models
```

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor,
    GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
from sklearn.neural_network import MLPRegressor

# Classification models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeRegressor
from sklearn.naive_bayes import GaussianNB

# Regression metrics
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    r2_score
)

# Classification metrics
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    precision_recall_curve,
    auc
)
# clustering
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# Download latest version
path = kagglehub.dataset_download("rakeshkapilavai/ai-tool-usage-
    by-indian-college-students-2025")
print("Path to dataset files:", path)
```

2.2 Data loading and Dataset overview

2.2.1 Data loading from kaggle

The dataset is loaded directly from Kaggle using the kagglehub library. The data is read into a pandas DataFrame to allow efficient manipulation and analysis throughout the

study.

```
data_path = os.path.join(path, "Students.csv")
data = pd.read_csv(data_path)
data_original = data.copy(deep=True)
```

2.2.2 Data overview

This step provides an initial overview of the dataset structure, including column names, data types, and general information about the variables.

```
display(data.columns)
```

Index(['Student_Name', 'College_Name', 'Stream', 'Year_of_Study', 'AI_Tools_Used', 'Daily_Usage_Hours', 'Use_Cases', 'Trust_in_AI_Tools', 'Impact_on_Grades', 'Do_Professors_Allow_Use','Preferred_AI_Tool', 'Awareness_Level', 'Willing_to_Pay_for_Access', 'State', 'Device_Used', 'Internet_Access'],
dtype='object')

```
display(data.info())
```

Table 2.1: Dataset structure and data types

#	Column Name	Non-Null Count	Data Type
0	Student_Name	3614	object
1	College_Name	3614	object
2	Stream	3614	object
3	Year_of_Study	3614	int64
4	AI_Tools_Used	3614	object
5	Daily_Usage_Hours	3614	float64
6	Use_Cases	3614	object
7	Trust_in_AI_Tools	3614	int64
8	Impact_on_Grades	3614	int64
9	Do_Professors_Allow_Use	3614	object
10	Preferred_AI_Tool	3614	object
11	Awareness_Level	3614	int64
12	Willing_to_Pay_for_Access	3614	object
13	State	2000	object
14	Device_Used	3614	object
15	Internet_Access	3614	object

```
data.describe()
```

Table 2.2: Statistical summary of numerical variables

Statistic	Year of Study	Daily Usage Hours	Trust in AI Tools	Impact on Grades	Awareness Level
Count	3614	3614	3614	3614	3614
Mean	2.47	2.56	3.02	0.00	5.83
Std	1.11	1.21	1.44	2.37	2.93
Min	1.00	0.50	1.00	-5.00	1.00
25%	2.00	1.50	2.00	-2.00	3.00
50% (Median)	2.00	2.60	3.00	0.00	6.00
75%	3.00	3.60	4.00	2.00	8.00
Max	4.00	5.00	5.00	5.00	10.00

Table 2.2 presents descriptive statistics for numerical variables, including central tendency and dispersion measures, providing insight into data distribution.

2.3 Data Cleaning

Data cleaning is a critical step in the data analysis process, as it directly affects the reliability and validity of the results.

2.3.1 Handling Missing Values

This step identifies missing values across all features to assess data completeness and determine appropriate cleaning strategies.

```
data.isna().sum()
```

Table 2.3: Number of missing values per feature

Feature	**Missing Values**
Student_Name	0
College_Name	0
Stream	0
Year_of_Study	0
AI_Tools_Used	0
Daily_Usage_Hours	0
Use_Cases	0
Trust_in_AI_Tools	0
Impact_on_Grades	0
Do_Professors_Allow_Use	0
Preferred_AI_Tool	0
Awareness_Level	0
Willing_to_Pay_for_Access	0
State	1614
Device_Used	0
Internet_Access	0

```
missing_percentage = data_original[data_original.State.isna()].
   shape[0] / data_original.shape[0] * 100
missing_percentage
```

44.659656889872714

```
sns.heatmap(data.isnull(),cbar=False)
plt.show()
```

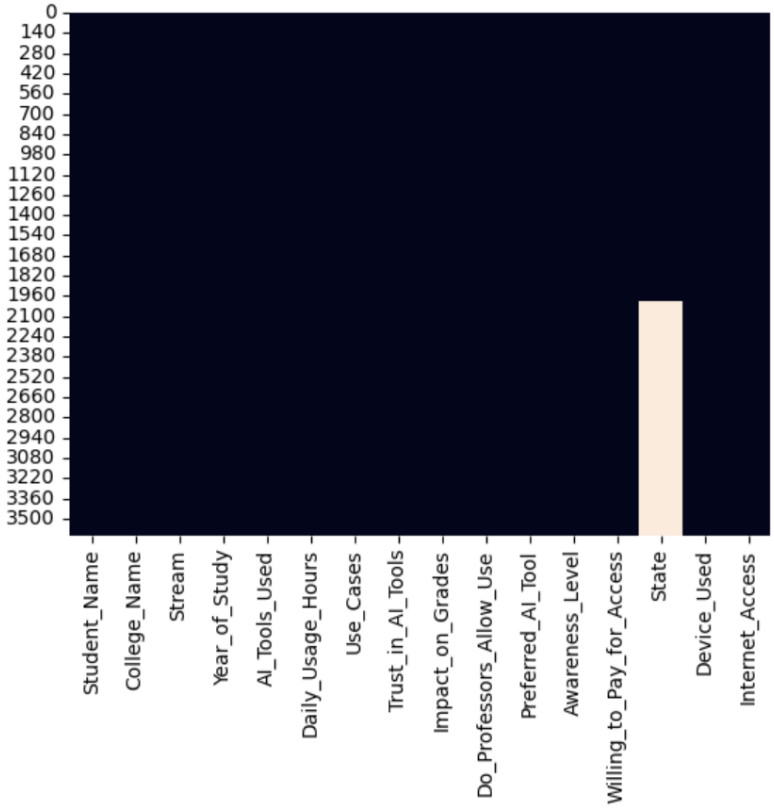


Figure 2.1: Missing data visualization

The results indicate that the feature "State" contains a significant proportion of missing values (approximately 45 percent), making it unsuitable for reliable analysis.

```
data = data.drop(axis=1, inplace=False, columns=['State'])
```

```
sns.heatmap(data.isnull())
plt.show()
```

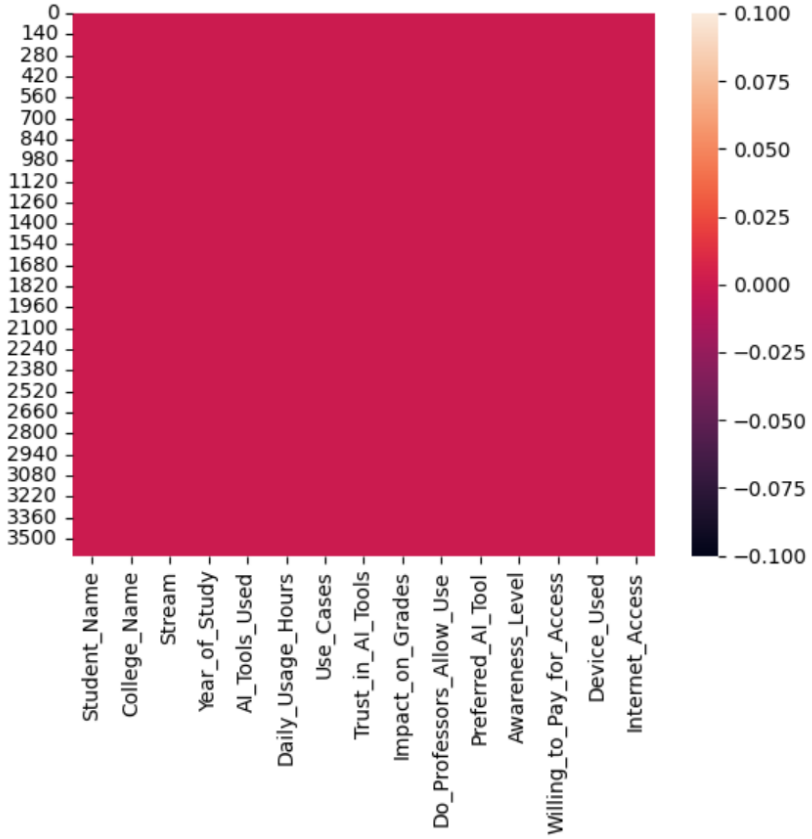


Figure 2.2: Missing data visualization after cleaning

Due to the high percentage of missing values, the "State" column is removed to maintain data quality and consistency.

2.3.2 Removing Duplicate Records

Duplicate records are identified and removed to prevent bias and ensure that each observation contributes uniquely to the analysis.

```
data.duplicated().sum()
```

np.int64(3)

```
data.drop_duplicates(inplace=True)
```

np.int64(0)

2.4 Univariate Analysis

2.4.1 Distribution of the Target Variable

The distribution of the target variable shows a concentration around neutral values, suggesting that AI tools neither strongly improve nor harm grades for most students. This balanced distribution is suitable for further modeling.

```
sns.displot(data, x='Impact_on_Grades', kind='kde')
```

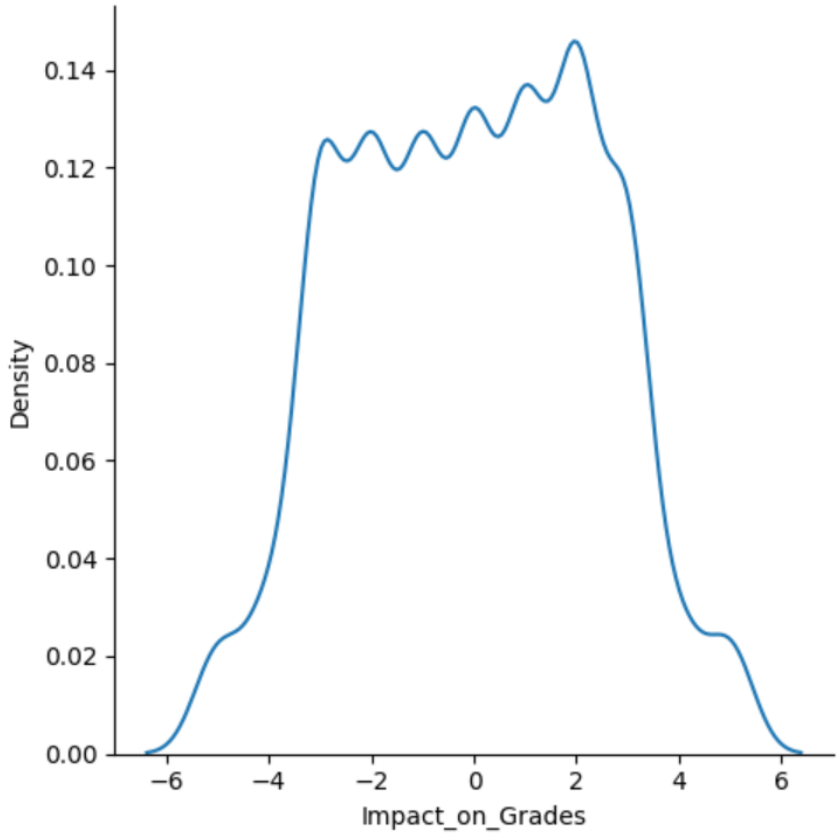


Figure 2.3: density distribution of the target variable.

Figure 2.3 confirms a relatively symmetric distribution, indicating the absence of strong class imbalance in the target variable.

2.4.2 Distribution of Numerical Features (Histograms)

Daily AI usage hours display moderate dispersion, with most students using AI tools between 1.5 and 3.5 hours per day. Extreme usage values are relatively rare.

```
sns.displot(data, x= 'Daily_Usage_Hours' , kind='kde')
```

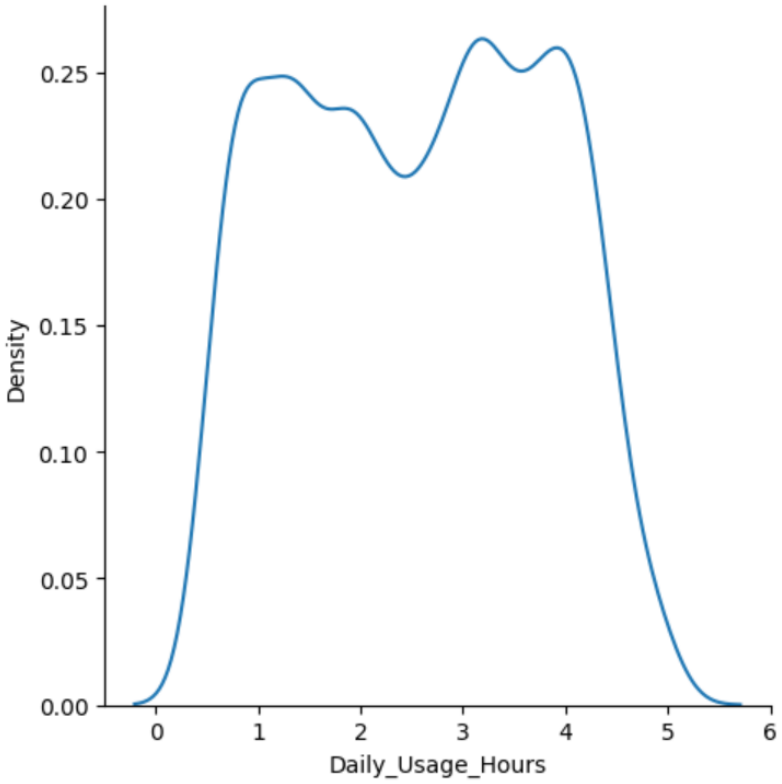


Figure 2.4: Density of daily usage

Figure 2.4 suggests a unimodal distribution, indicating a common usage pattern among students.

2.4.3 Distribution of Categorical Features

```
count_AI_used =data['AI_Tools_Used'].value_counts()
count_AI_used.plot(kind='barh')
```

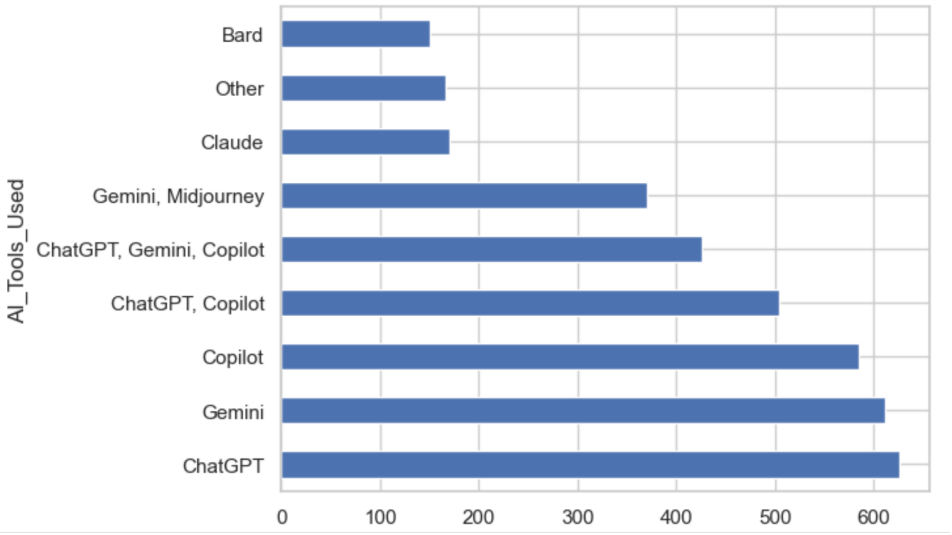


Figure 2.5: AI tools usage

The distribution of AI tools used reveals a strong dominance of generative AI platforms, particularly conversational and content-generation tools. This indicates a widespread

adoption of AI technologies that directly support learning, problem-solving, and content creation tasks.

The preference for specific tools suggests that ease of use, accessibility, and perceived usefulness play a significant role in students' adoption choices.

2.4.4 Outlier Detection

```
def detect_outlier(series: pd.Series):
    mu = np.mean(series)
    sigma = np.std(series)
    z_scores = (series - mu)/sigma
    outliers = series[ abs(z_scores) > 3]
    return outliers
```

```
numerical_columns = data_original.select_dtypes(include="number")
for col in numerical_columns.columns:
   n = detect_outlier(data_original[col]).shape[0]
   print(col, " ", str(n))
```

```
Year_of_Study   0
Daily_Usage_Hours   0
Trust_in_AI_Tools   0
Impact_on_Grades   0
Awareness_Level   0
```

Outlier detection using the Z-score method reveals no extreme values in numerical features.
This indicates stable usage patterns and reduces the need for aggressive data correction.

2.5 Multivariate Analysis

2.5.1 Relationship Between Numerical Features(Scatter/Line Plots)

```
mean_poor = data[data['Internet_Access'] == "Poor"]['
   Daily_Usage_Hours'].mean()
mean_medium = data[data['Internet_Access'] == "Medium"]['
   Daily_Usage_Hours'].mean()
mean_high = data[data['Internet_Access'] == "High"]['
   Daily_Usage_Hours'].mean()

display(mean_poor, mean_medium, mean_high)
sns.barplot(data, x='Internet_Access', y='Daily_Usage_Hours')
```

```
np.float64(2.5291869918699184)
np.float64(2.60429252782194)
np.float64(2.542564559216385)
```

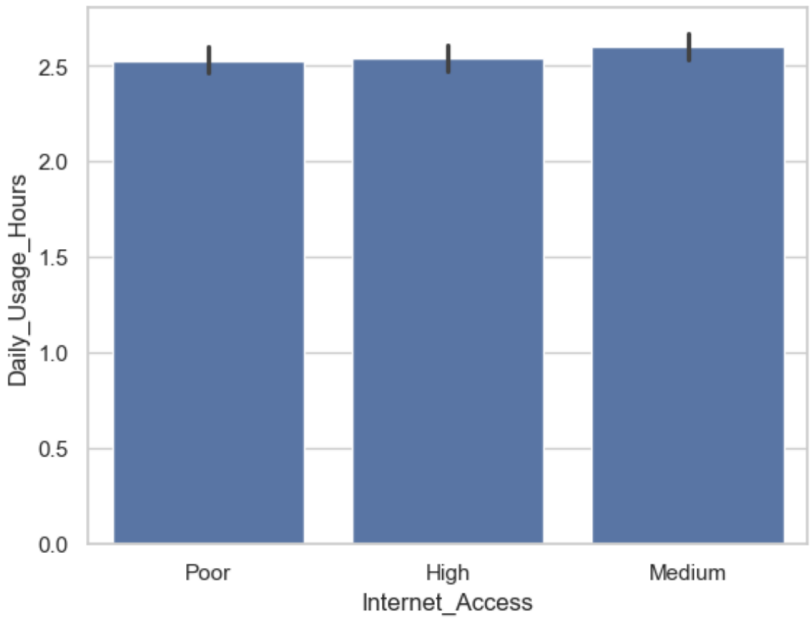


Figure 2.6: impact of access to internet on daily usage

The comparison of daily AI usage across different levels of internet access shows only minor variations between groups. Although students with medium and high internet access display slightly higher average usage, the differences remain marginal.

This observation suggests that once a minimum level of internet connectivity is available, access quality does not significantly constrain AI tool usage among students.

2.5.2 Analysis of Features vs Target Variable

```
sns.barplot(data, x='Impact_on_Grades', y='Daily_Usage_Hours')
```

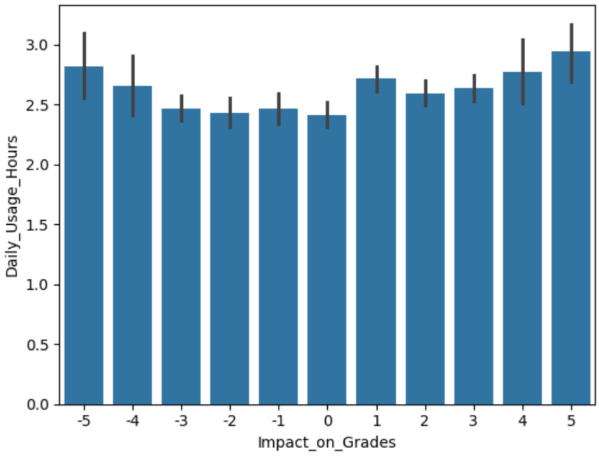


Figure 2.7: Impact of using AI on grades

The relationship between daily AI usage hours and the perceived impact on grades appears to be non-linear. Moderate AI usage is associated with neutral to slightly positive academic outcomes.

However, increased usage beyond a certain point does not systematically translate into improved grades. This finding highlights that the effectiveness of AI tools depends more on how they are used rather than the duration of usage alone.

2.5.3 Correlation Matrix and Heatmap

```
coor=data[["Year_of_Study","Trust_in_AI_Tools","Awareness_Level"
    ]].corr()
plt.figure(figsize=(12, 10))
sns.heatmap(coor, annot=True, cmap='RdBu_r', center=0, fmt=".2f")
plt.title("Correlation Matrix")
plt.show()
```

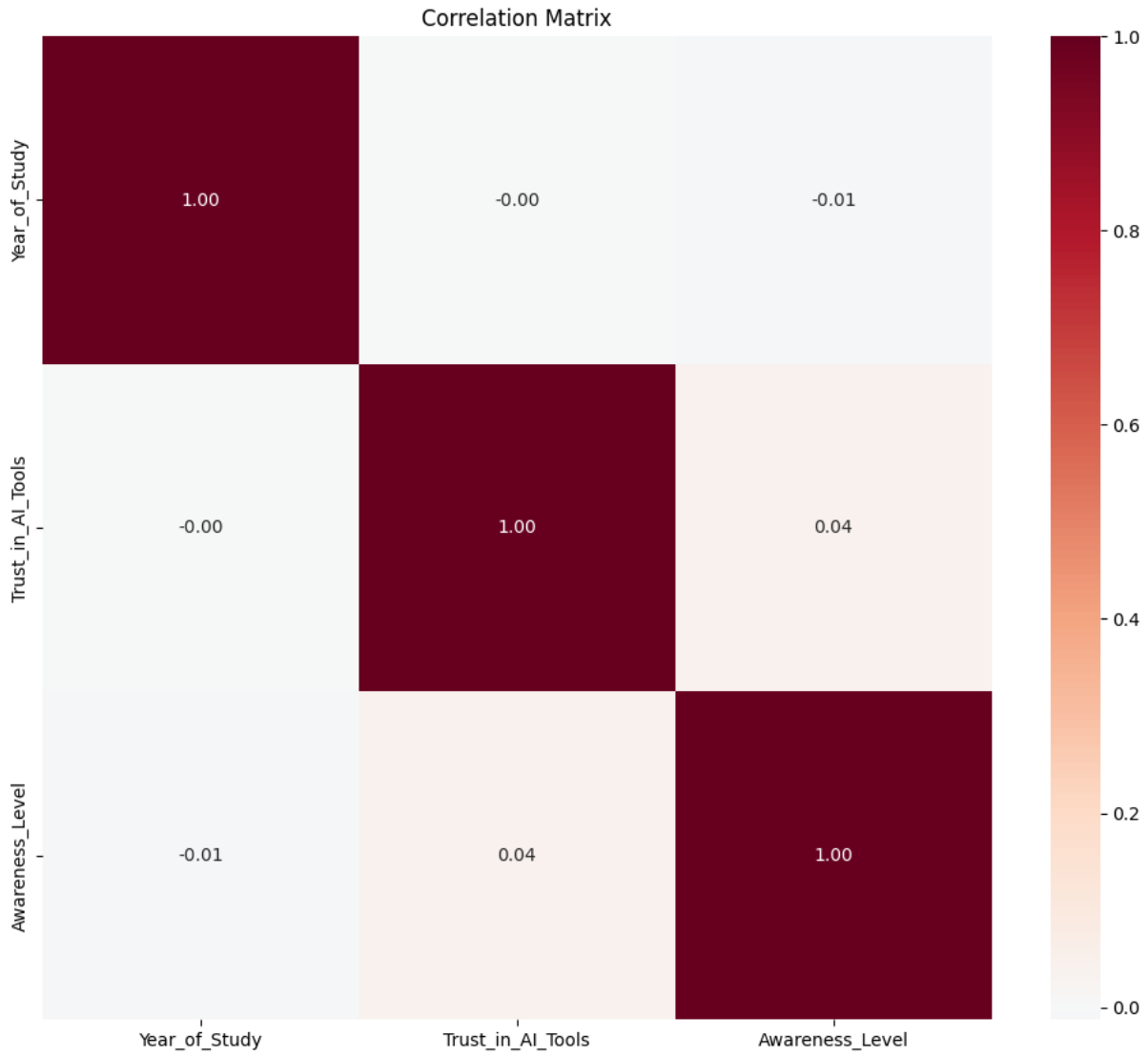


Figure 2.8: Correlation Matrix of Selected Numerical Variables

The correlation matrix shows weak linear relationships between year of study, trust in AI, and awareness level. This suggests that these factors influence AI usage independently.

Chapter 3

Data Preparation for Modeling

The data preparation phase is the pivotal stage where raw, heterogeneous information is transformed into an optimized numerical structure for machine learning. This section outlines the methodological justifications for the techniques employed to ensure the statistical integrity of the model.

3.1 Encoding Strategies and Technical Justifications

Encoding is the process by which qualitative variables are converted into numerical values. The choice of encoding method is dictated by the intrinsic nature of each variable to avoid altering the data's underlying geometry.

3.1.1 Multi-Label Binarization (MLB) – Handling Multi-choice data

For variables such as Use_Cases and AI_Tools_Used, where a student can select multiple responses simultaneously, the use of MultiLabelBinarizer (MLB) was essential.

```
data['Use_Case_list'] = data['Use_Cases'].str.strip().str.split(',').apply(lambda x: ["Use_Cases_"+t.strip() for t in x])
# display(data.Use_Case_list.dtype)
# display(data.Use_Case_list.head())
mlb = MultiLabelBinarizer()
use_cases_encoded = pd.DataFrame(
    mlb.fit_transform(data["Use_Case_list"]),
    columns=mlb.classes_,
    index=data.index
)

use_cases_encoded["Use_Cases_Exam_Prep"] = (
    use_cases_encoded["Use_Cases_Exam Prep"] | use_cases_encoded["Use_Cases_Exam Preparation"]
).astype(int)

use_cases_encoded["Use_Cases_Projects"] = (
    use_cases_encoded["Use_Cases_Project Work"] | use_cases_encoded["Use_Cases_Projects"]
).astype(int)

use_cases_encoded.drop(
```

```
    columns=[
        "Use_Cases_Exam Prep",
        "Use_Cases_Exam Preparation",
        "Use_Cases_Project Work",
        "Use_Cases_Projects",
    ],
    inplace=True
)

data['Use_Case_Count'] = use_cases_encoded.sum(axis=1)
data = pd.concat([data, use_cases_encoded], axis=1)

display(use_cases_encoded.columns.shape)
display(use_cases_encoded.columns)
display(use_cases_encoded.head())
```

```
(9,)
Index(['Use_Cases_Assignments', 'Use_Cases_Coding Help',
       'Use_Cases_Content Writing', 'Use_Cases_Doubt Solving',
       'Use_Cases_Learning new topics', 'Use_Cases_MCQ Practice',
       'Use_Cases_Notes', 'Use_Cases_Resume Writing', 'Use_Cases_Exam_Prep'],
      dtype='object')
```

Table 3.1: Use Cases Binary Encoded

ID	Assignments	Coding	Content Writing	Doubt Solving	Learning New Topics	MCQ Practice	Notes	Resume Writing	Exam Prep
0	1	1	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0
2	0	0	0	0	0	1	0	0	0
3	0	0	1	0	0	0	0	0	0
4	0	0	0	1	0	0	0	1	0

```
data['AI_Tools_Used_list'] = data['AI_Tools_Used'].str.strip().
    str.split(',').apply(lambda x: ["AI_Tools_Used_"+ t.strip() for
     t in x])
# display(data.Use_Case_list.dtype)
# display(data.Use_Case_list.head())
mlb = MultiLabelBinarizer()
ai_tools_used = pd.DataFrame(
    mlb.fit_transform(data["AI_Tools_Used_list"]),
    columns=mlb.classes_,
    index=data.index
)

data['AI_Tools_Count'] = use_cases_encoded.sum(axis=1)
data = pd.concat([data, ai_tools_used], axis=1)

display(ai_tools_used.columns.shape)
display(ai_tools_used.columns)
display(ai_tools_used.head())
```

```
(7,)
```

```
Index(['AI_Tools_Used_Bard', 'AI_Tools_Used_ChatGPT', 'AI_Tools_Used_Claude',
       'AI_Tools_Used_Copilot', 'AI_Tools_Used_Gemini',
       'AI_Tools_Used_Midjourney', 'AI_Tools_Used_Other'],
      dtype='object')
```

Table 3.2: AI Tools Usage Binary Encoded

ID	Bard	ChatGPT	Claude	Copilot	Gemini	Midjourney	Other
0	0	0	0	0	1	0	0
1	0	1	0	0	0	0	0
2	0	0	0	1	0	0	0
3	0	0	0	1	0	0	0
4	0	0	0	0	1	0	0

Tables 3.1 and 3.2 present the binary encoding of multi-choice variables using the Multi-Label Binarization technique. Each column corresponds to a specific use case or AI tool, while binary values indicate whether a student selected the corresponding option. This encoding preserves the multi-response nature of the data and allows simultaneous representation of multiple use cases and tools for a single student, making the dataset suitable for machine learning models without introducing artificial ordering.

3.1.2 One-Hot Encoding (OHE) – Maintaining Neutrality

Nominal variables such as Stream or Device_Used were processed using One-Hot Encoding. This choice is based on the necessity of maintaining mathematical neutrality between categories.

```
ohe = OneHotEncoder(sparse_output=False, drop='first')
encoded = ohe.fit_transform(data[['Stream']])
df_encoded = pd.DataFrame(encoded, columns=ohe.
   get_feature_names_out(['Stream']))
data = pd.concat([data, df_encoded], axis=1)

ohe = OneHotEncoder(sparse_output=False, drop='first')
encoded = ohe.fit_transform(data[['Preferred_AI_Tool']])
df_encoded = pd.DataFrame(encoded, columns=ohe.
   get_feature_names_out(['Preferred_AI_Tool']))
data = pd.concat([data, df_encoded], axis=1)

ohe = OneHotEncoder(sparse_output=False, drop='first')
encoded = ohe.fit_transform(data[['Device_Used']])
df_encoded = pd.DataFrame(encoded, columns=ohe.
   get_feature_names_out(['Device_Used']))
data = pd.concat([data, df_encoded], axis=1)

data_original.head()
```

```
Index(['Student_Name', 'College_Name', 'Stream', 'Year_of_Study',
       'AI_Tools_Used', 'Daily_Usage_Hours', 'Use_Cases', 'Trust_in_AI_Tools',
       'Impact_on_Grades', 'Do_Professors_Allow_Use',
```

```
       ...
       'Stream_nan', 'Preferred_AI_Tool_ChatGPT', 'Preferred_AI_Tool_Claude',
       'Preferred_AI_Tool_Copilot', 'Preferred_AI_Tool_Gemini',
       'Preferred_AI_Tool_Other', 'Preferred_AI_Tool_nan',
       'Device_Used_Mobile', 'Device_Used_Tablet', 'Device_Used_nan'],
      dtype='object', length=145)
```

3.1.3 Ordinal Encoding and Preserving Hierarchical Integrity

The Internet Access variable presents a natural ordinal structure ranging from Poor to High. To preserve this hierarchy, Ordinal Encoding is applied instead of One-Hot Encoding.
The encoded values are subsequently normalized using Min-Max Scaling to confine them within the [0, 1] interval. This ensures that the relative distances between categories are respected while maintaining numerical stability for machine learning algorithms.

```
ordinalEncoder = OrdinalEncoder(categories=[["Poor", "Medium", "
    High"]])
data.dropna(inplace=True)
data['Internet_Access_Encoded'] = MinMaxScaler().fit_transform(
    ordinalEncoder.fit_transform(data[['Internet_Access']]) )
data.Internet_Access_Encoded.head()
```

Table 3.3: Internet Access Encoding

ID	Internet Access Encoded
0	0.0
1	0.0
2	0.0
3	1.0
4	0.5

```
data['Do_Professors_Allow_Use_encoded'] = LabelBinarizer().
    fit_transform(data['Do_Professors_Allow_Use'])
data['Willing_to_Pay_for_Access_encoded'] = LabelBinarizer().
    fit_transform(data['Willing_to_Pay_for_Access'])
display(data.Do_Professors_Allow_Use_encoded.head())
print("*"*200)
display(data.Willing_to_Pay_for_Access_encoded.head())
```

Table 3.4: Binary Encoding of Professors' Permission and Willingness to Pay

ID	Professors Allow Use	Willing to Pay for Access
0	0	1
1	1	0
2	0	0
3	1	0
4	1	1

Tables 3.3 and 3.4 summarize the encoding strategies applied to binary and ordinal variables. These transformations ensure that qualitative information is converted into meaningful numerical representations without distorting the original semantics of the data.

3.2 Data Scaling

Rescaling data is imperative to harmonize the influence of each variable on the model's cost function.

- Standardization (StandardScaler): This was specifically applied to the Daily_Usage_Hours variable. By centering the data around a mean of 0 with a standard deviation of 1, we enable the algorithm to easily detect atypical users (outliers) without suppressing the subtle nuances between moderate users.
- Min-Max Normalization (MinMaxScaler): This was preferred for variables derived from rating scales, such as Trust_in_AI_Tools. This method is ideal as it compresses values into the [0, 1] interval while scrupulously preserving the original structure and relative distances between scores. A rating of 10 remains the absolute maximum (1.0), which is crucial for the accurate interpretation of survey data.

```
sns.boxplot(y=data['Year_of_Study'])
plt.show()
sns.boxplot(y=data['Daily_Usage_Hours'])
plt.show()
sns.boxplot(y=data['Trust_in_AI_Tools'])
plt.show()
sns.boxplot(y=data['Awareness_Level'])
plt.show()
```

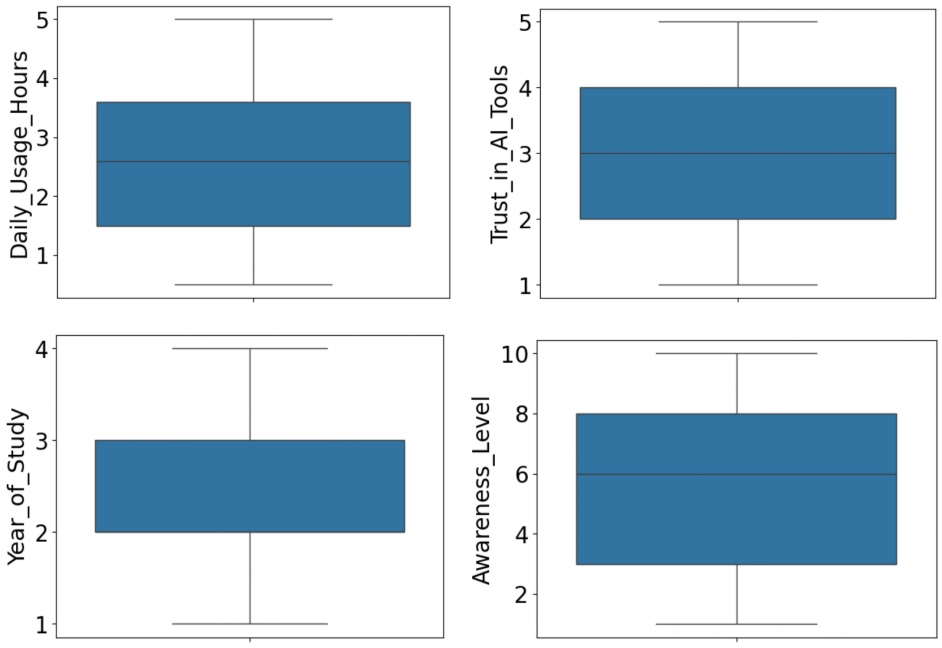


Figure 3.1: Boxplots of Key Numerical Variables Before Scaling

The box plots presented in Figure 3.1 illustrate the distribution of key numerical variables before scaling. While the variables exhibit different ranges and dispersions, no extreme anomalies are observed.

These visualizations confirm the necessity of scaling to prevent variables with larger numeric ranges from disproportionately influencing the learning algorithms.

```
data['Year_of_Study_encoded'] = MinMaxScaler().fit_transform(data
   [['Year_of_Study']])
data['Daily_Usage_Hours_encoded'] = StandardScaler().
   fit_transform(data[['Daily_Usage_Hours']])
data['Trust_in_AI_Tools_encoded'] = MinMaxScaler().fit_transform(
   data[['Trust_in_AI_Tools']])
data['Awareness_Level_encoded'] = MinMaxScaler().fit_transform(
   data[['Awareness_Level']])
```

After applying the selected scaling techniques, all numerical features are transformed into comparable ranges. Standardization is used for Daily Usage Hours to highlight deviations from average usage, while Min-Max normalization preserves the relative structure of survey-based rating variables.

3.3 Feature Selection and Correlation Analysis

```
data.columns
```

```
Index(['Student_Name', 'College_Name', 'Stream', 'Year_of_Study',
       'AI_Tools_Used', 'Daily_Usage_Hours', 'Use_Cases', 'Trust_in_AI_Tools',
       'Impact_on_Grades', 'Do_Professors_Allow_Use', 'Preferred_AI_Tool',
       'Awareness_Level', 'Willing_to_Pay_for_Access', 'Device_Used',
       'Internet_Access', 'Use_Case_list', 'Use_Case_Count',
       'Use_Cases_Assignments', 'Use_Cases_Coding Help',
       'Use_Cases_Content Writing', 'Use_Cases_Doubt Solving',
       'Use_Cases_Learning new topics', 'Use_Cases_MCQ Practice',
       'Use_Cases_Notes', 'Use_Cases_Resume Writing', 'Use_Cases_Exam_Prep',
       'AI_Tools_Used_list', 'AI_Tools_Count', 'AI_Tools_Used_Bard',
       'AI_Tools_Used_ChatGPT', 'AI_Tools_Used_Claude',
       'AI_Tools_Used_Copilot', 'AI_Tools_Used_Gemini',
       'AI_Tools_Used_Midjourney', 'AI_Tools_Used_Other', 'Stream_Arts',
       'Stream_Commerce', 'Stream_Engineering', 'Stream_Hotel-management',
       'Stream_Law', 'Stream_Management', 'Stream_Medical', 'Stream_Pharmacy',
       'Stream_Science', 'Preferred_AI_Tool_ChatGPT',
       'Preferred_AI_Tool_Claude', 'Preferred_AI_Tool_Copilot',
       'Preferred_AI_Tool_Gemini', 'Preferred_AI_Tool_Other',
       'Preferred_AI_Tool_nan', 'Device_Used_Mobile', 'Device_Used_Tablet',
       'Device_Used_nan', 'Internet_Access_Encoded',
       'Do_Professors_Allow_Use_encoded', 'Willing_to_Pay_for_Access_encoded',
       'Year_of_Study_encoded', 'Daily_Usage_Hours_encoded',
       'Trust_in_AI_Tools_encoded', 'Awareness_Level_encoded'],
      dtype='object')
```

```
data_encoded = data.select_dtypes(include='number').copy(deep=
    True)
data_encoded.drop(columns= ['Year_of_Study', 'Daily_Usage_Hours',
     'Trust_in_AI_Tools', 'Awareness_Level'])
```

After encoding and scaling, numerical features are selected for correlation analysis. Original non-encoded variables are retained in the dataset but are not used as predictors in the modeling phase.

```
target_corr = data_encoded.corr()['Impact_on_Grades'].sort_values
    (ascending=False)
target_corr_disp = target_corr[ np.abs(target_corr) > 0.03]

plt.figure(figsize=(10, 8))
sns.barplot(x=target_corr_disp.values, y=target_corr_disp.index,
    palette='coolwarm')
plt.title('Correlation with Impact on Grades')
plt.show()
```

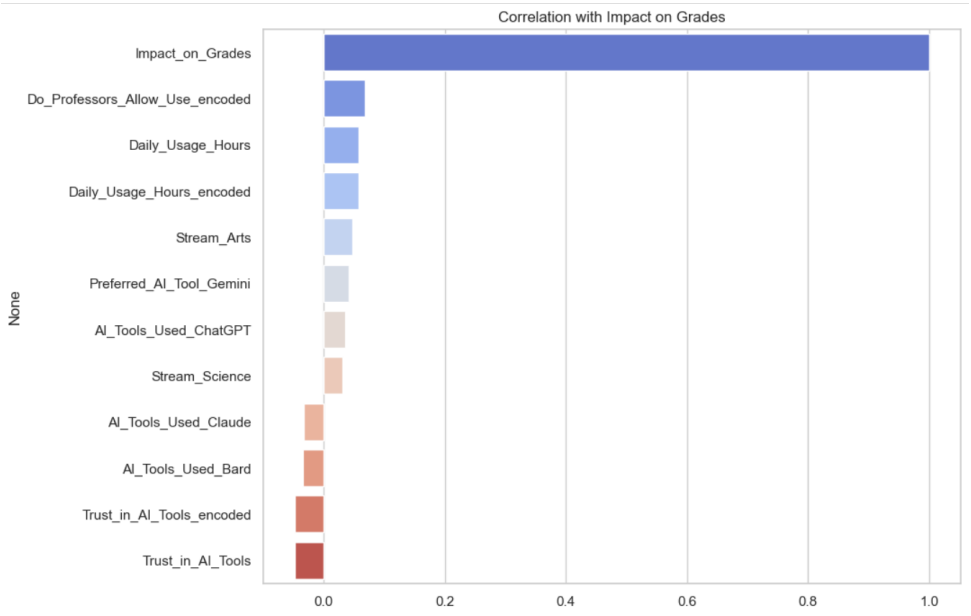


Figure 3.2: Correlation of Encoded Features with Academic Impact

Figure 3.2 highlights the features that exhibit the strongest correlations with the target variable Impact on Grades. Although the correlation coefficients remain relatively small, several variables demonstrate noticeable influence.

This observation suggests that academic impact is a multifactorial phenomenon influenced by multiple weak predictors rather than a single dominant feature.

```
top_features = target_corr[abs(target_corr) > 0.03].index
corr_matrix = data_encoded[top_features].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, cmap='RdBu_r', center=0, fmt
    =".2f")
plt.title("Correlation Matrix")
plt.show()
```

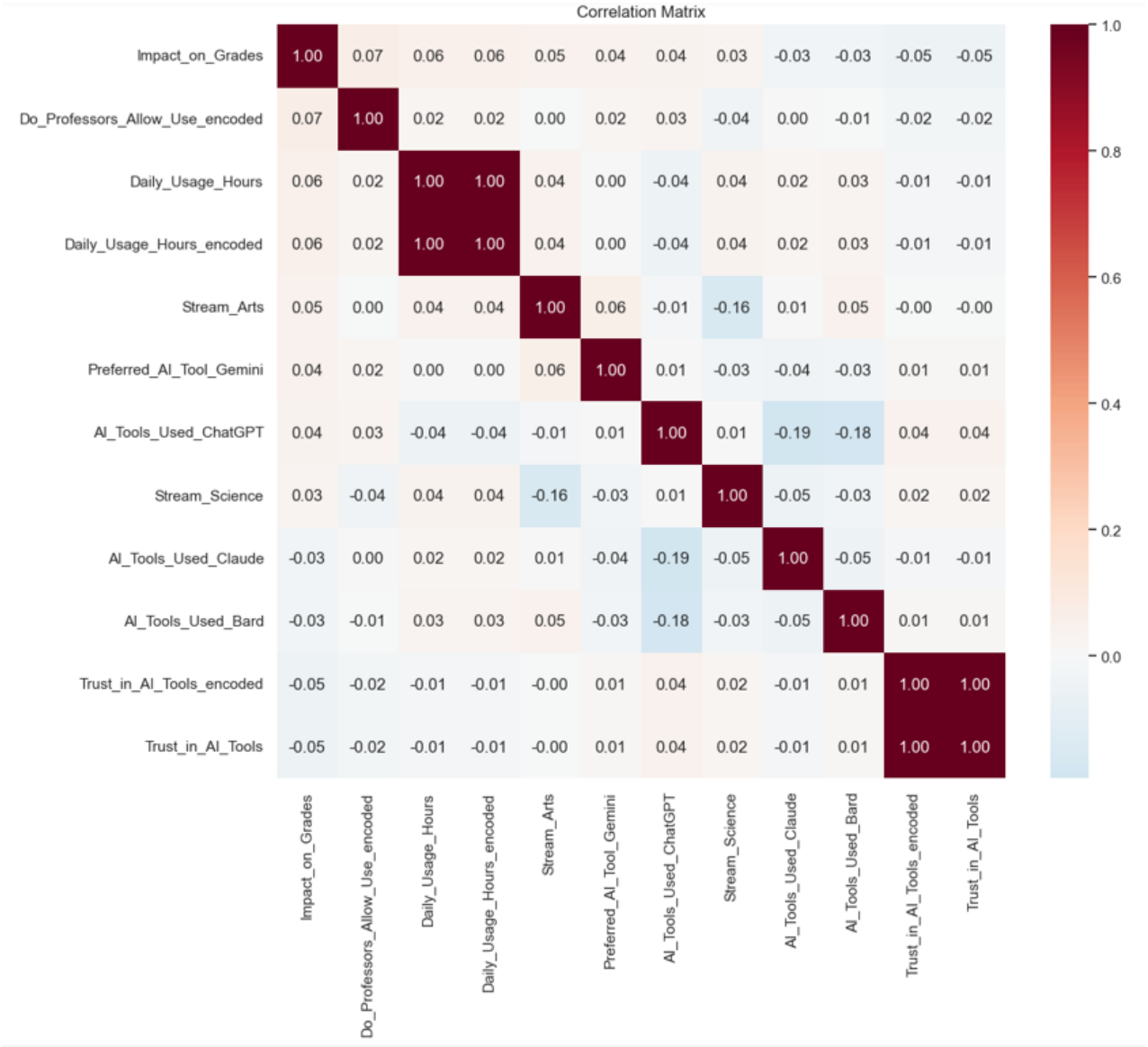


Figure 3.3: Heatmap of Correlations Among Selected Significant Features

The correlation heatmap in Figure 3.3 provides a global view of the relationships among the most relevant features. The absence of strong correlations between predictors confirms limited multicollinearity, which is desirable for both interpretability and model robustness.

3.4 Validation via P-Value: Scientific Proof

To complement correlation analysis, statistical validation is performed using Pearson's correlation test. This approach evaluates not only the strength but also the statistical significance of relationships between each feature and the target variable.

Features with p-values below the 0.05 threshold are considered statistically significant, providing scientific justification for their inclusion in predictive modeling.

```
from scipy.stats import pearsonr
import pandas as pd

# Initialize a list to store correlation results
correlation_results = []
```

```
6
7  # Define the target variable
8  target = 'Impact_on_Grades'
9
10 # Iterate through all columns in the encoded dataframe
11 for col in data_encoded.columns:
12     if col != target:
13         # Calculate Pearson correlation and p-value
14         # pearsonr returns (correlation_coefficient, p_value)
15         corr, p_val = pearsonr(data_encoded[target], data_encoded
               [col])
16
17         correlation_results.append({
18             'Feature': col,
19             'Correlation': round(corr, 4),
20             'P-Value': round(p_val, 6),
21             'Is_Significant': p_val < 0.05
22         })
23 # Create a DataFrame to display the results
24 significance_df = pd.DataFrame(correlation_results)
25
26 # Sort by p-value to see the most statistically significant
       features first
27 significance_df = significance_df.sort_values(by='P-Value',
       ascending=True)
28
29 # Display only the significant features
30 significant_features = significance_df[significance_df['
       Is_Significant'] == True ]
31 print("Statistically Significant Features (p < 0.05):")
32 display(significant_features)
```

Table 3.5: Statistically Significant Correlations

Feature	Correlation	P-Value	Significant
Do_Professors_Allow_Use_encoded	0.0670	0.000057	True
Daily_Usage_Hours	0.0579	0.000503	True
Daily_Usage_Hours_encoded	0.0579	0.000503	True
Trust_in_AI_Tools	-0.0474	0.004420	True
Trust_in_AI_Tools_encoded	-0.0474	0.004420	True
Preferred_AI_Tool_Gemini	0.0428	0.010132	True
Stream_Arts	0.0400	0.016355	True
AI_Tools_Used_ChatGPT	0.0370	0.026301	True
Stream_Science	0.0355	0.033028	True
AI_Tools_Used_Bard	-0.0341	0.040528	True
AI_Tools_Used_Claude	-0.0333	0.045684	True

Although several variables exhibit statistically significant correlations with the target variable Impact_on_Grades, the magnitude of these correlations remains weak. In particular, daily AI usage hours and professors' permission show positive associations with academic impact, while trust in AI tools presents a weak negative correlation. These

results should not be interpreted as causal relationships, but rather as indications of differing usage behaviors and perceptions among students. Overall, these variables explain only a limited portion of the variance in academic impact, confirming that academic performance is influenced by multiple factors beyond AI usage.

```
sns.jointplot(data_encoded, x='Daily_Usage_Hours', y='
    Impact_on_Grades', kind='hex')
```

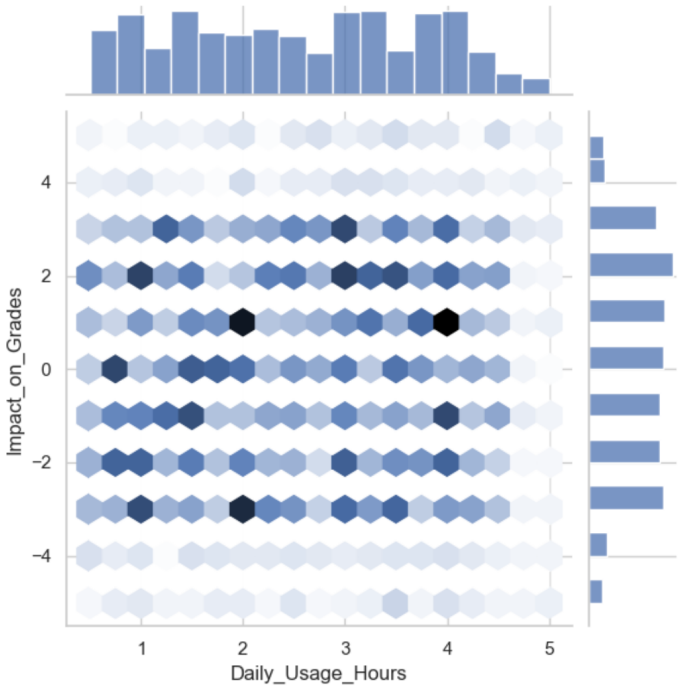


Figure 3.4: Joint Distribution of Daily AI Usage Hours and Academic Impact

The joint distribution in Figure 3.4 confirms the weak but consistent relationship between daily AI usage and academic impact. The density concentration around moderate usage levels reinforces the conclusion that excessive usage does not guarantee improved grades.

```
cols = use_cases_encoded.columns
counts = np.zeros(shape=cols.shape)
for i in range(cols.shape[0]) :
    counts[i] = data[cols[i]].sum()

df = pd.DataFrame({'labels': cols, 'counts': counts}).sort_values
    (by='counts')

plt.barh(df['labels'], df['counts'])
```

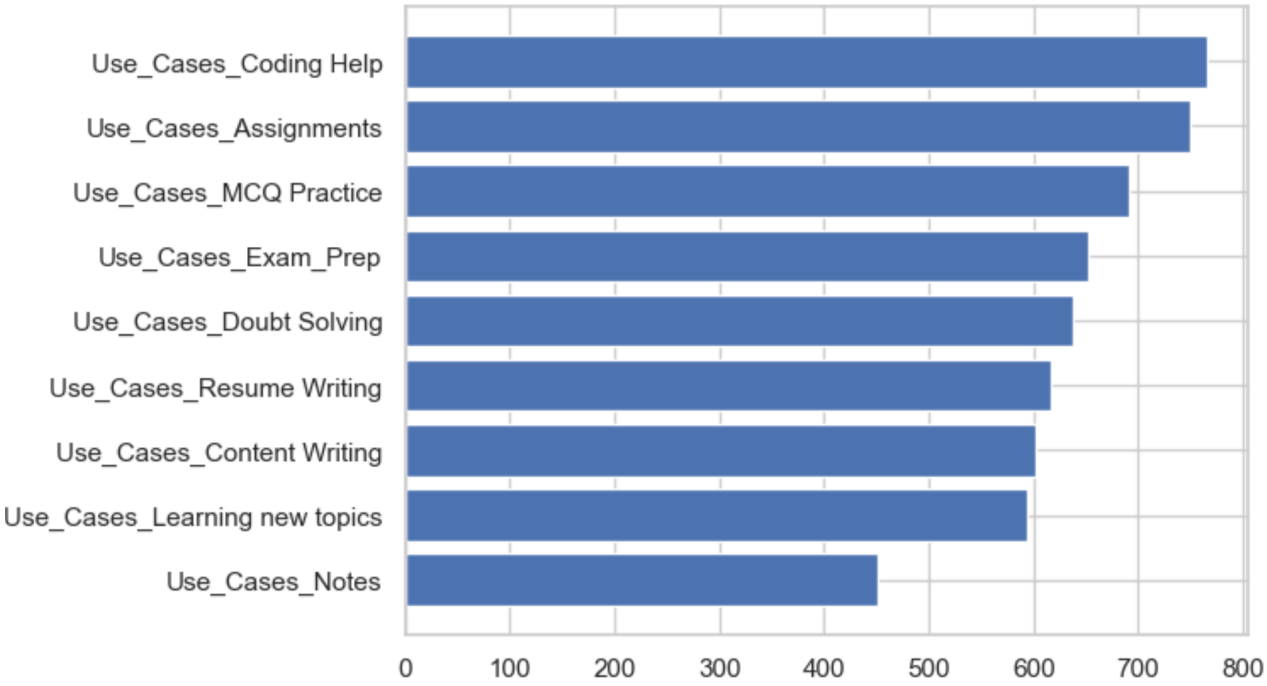


Figure 3.5: Frequency Distribution of AI Use Cases Among Students

Figure 3.5 illustrates the frequency of different AI use cases among students. Academic support activities such as coding help, assignments, and exam preparation dominate usage patterns, highlighting AI's role as a learning assistance tool rather than a replacement for core academic effort.

```
average_for_allowed = data[data['Do_Professors_Allow_Use'] == "
    Yes"]['Daily_Usage_Hours'].mean()
average_for_forbidden = data[data['Do_Professors_Allow_Use'] == "
    No"]['Daily_Usage_Hours'].mean()
display(average_for_allowed,average_for_forbidden)
sns.barplot(data, x='Do_Professors_Allow_Use', y='
    Daily_Usage_Hours')
```

```
np.float64(2.5884704519119355)\newline
np.float64(2.5336875664187035)
```

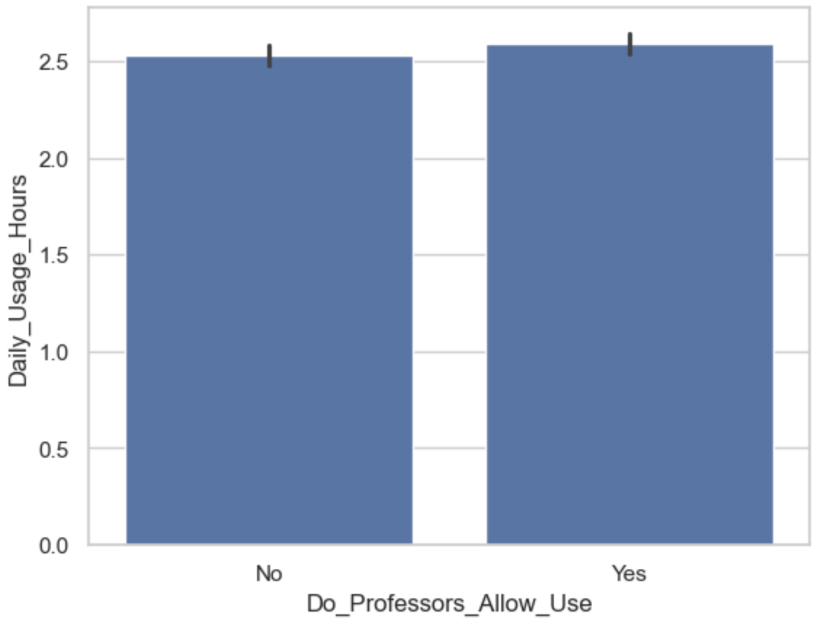


Figure 3.6: Comparison of Daily AI Usage Based on Professors' Permission

Students whose professors allow the use of AI tools exhibit slightly higher average daily usage. Although the difference is modest, it suggests that institutional approval may encourage more consistent and confident use of AI technologies in academic contexts.

3.5 Final Optimization

```
X = data_encoded.drop(columns=['Impact_on_Grades'])
y = data_encoded['Impact_on_Grades']
min_count = 0
display(min_count)
X = X.loc[:, (X.sum(axis=0) >= min_count)]
X.shape[1]
```

```
0
47
```

Since all selected features appear in at least one observation, no feature was removed at this stage. This step is kept for completeness and reproducibility of the pipeline.

3.6 Data Splitting (Training and Testing Sets)

3.6.1 Reasons for Splitting the Data

Splitting the dataset into training and testing sets is essential to evaluate the generalization capability of machine learning models. This separation ensures that model performance is assessed on unseen data, reducing the risk of overfitting and providing a realistic estimate of predictive accuracy.

3.6.2 Data Splitting Method

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Chapter 4

Classification Models

4.1 Multi-class Classification

4.1.1 DecisionTreeClassifier

The Decision Tree classifier is selected as an initial multi-class model due to its interpretability and ability to handle non-linear relationships.

Decision Trees recursively split the feature space based on criteria that maximize class separation, making them suitable for exploratory classification tasks where understanding decision logic is important.

4.1.2 Model Implementation

```
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
```

The Decision Tree model is trained using the training dataset with default hyperparameters. A fixed random state is used to ensure reproducibility of the results.

The trained model learns a set of hierarchical decision rules based on the selected features to predict the different levels of academic impact.

4.1.3 Results and Confusion Matrix

```
y_pred = model.predict(X_test)
y_true = y_test

acc = accuracy_score(y_true, y_pred)
display(acc)
cm = confusion_matrix(y_true, y_pred)
cmd = ConfusionMatrixDisplay(confusion_matrix=cm)
cmd.plot()
```

0.4903047091412742

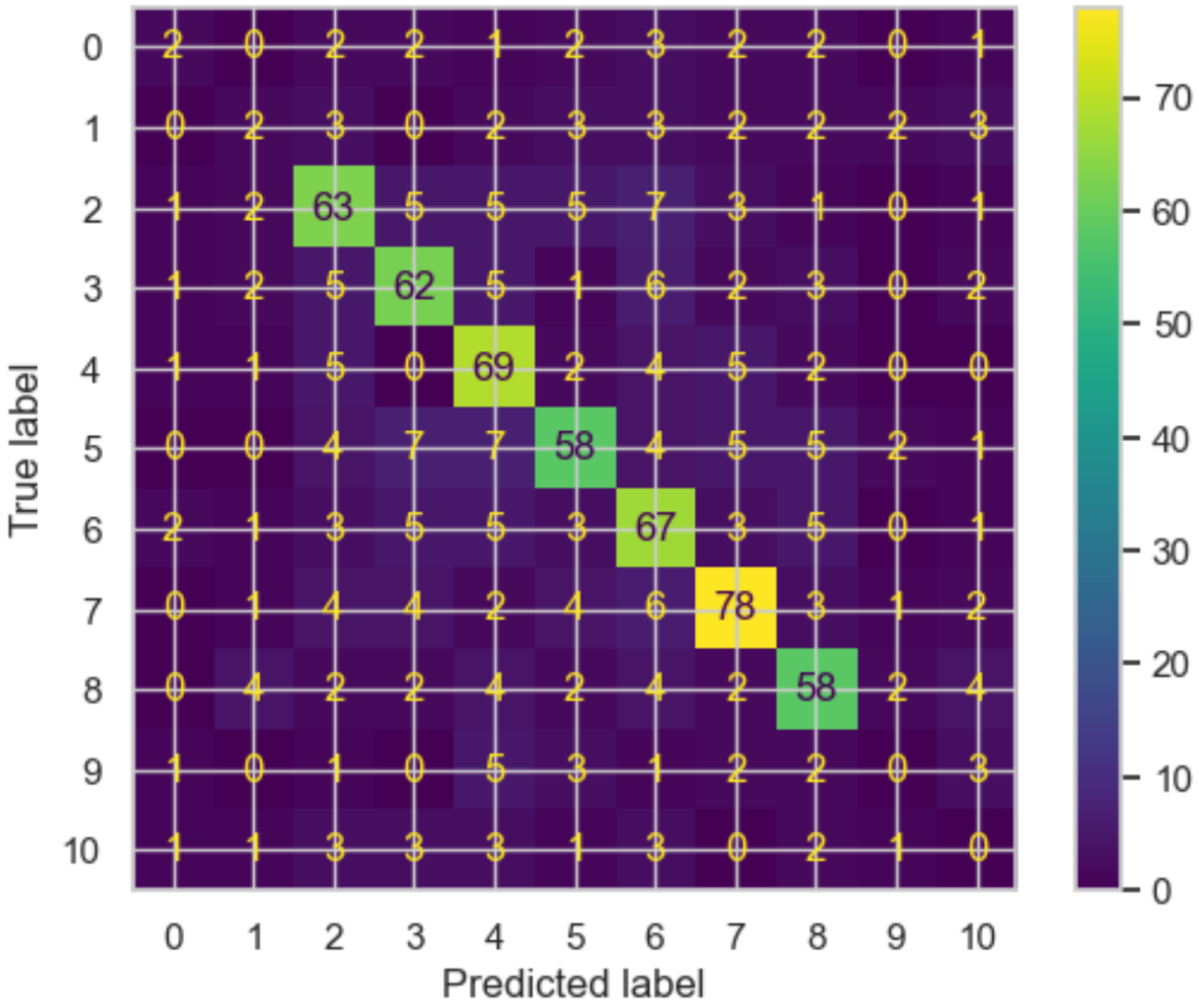


Figure 4.1: Confusion Matrix of the Multi-class Decision Tree Classifier

Figure 4.1 illustrates the confusion matrix of the Decision Tree classifier. The concentration of values along near-diagonal cells indicates partial predictive ability, while the dispersion across adjacent classes highlights ambiguity in students' perceived academic impact.

4.2 Binary classification

4.2.1 Definition of each Model

Multiple classification models are evaluated to compare their effectiveness in predicting academic impact.

Random Forest Classifier: An ensemble method that combines multiple decision trees to improve robustness and reduce overfitting.

Decision Tree Classifier: A single-tree model offering interpretability but potentially higher variance.

Support Vector Classifier (SVC): A margin-based classifier that seeks optimal separation between classes.

Gaussian Naive Bayes: A probabilistic model based on Bayes' theorem, assuming feature independence.

4.2.2 Model Implementation

```
models = {
    "RandomForest": RandomForestClassifier(random_state=42),
    "DecisionTreeClassifier": DecisionTreeClassifier(random_state
       =42),
    "SVC": SVC(),
    "Gaussian": GaussianNB(),
}
```

```
y_train_class = y_train.apply(lambda x : 1 if x > 0 else 0)
y_test_class = y_test.apply(lambda x : 1 if x > 0 else 0)
```

Each model is trained using the same training dataset to ensure fair comparison. Predictions are generated on the test set, and performance is evaluated using accuracy, F1-score, and recall metrics.

4.2.3 Results and Visualisation

```
results = []

# Initialize dictionary with the real values
comparison_dict = {"real_values": y_test_class.values.flatten()}

for name, model in models.items():
    # Fit and Predict
    model.fit(X_train, y_train_class)
    y_pred = model.predict(X_test)

    # Store metrics - added 'weighted' to handle multi-class
       tasks safely
    results.append({
        "model": name,
        "accuracy": accuracy_score(y_test_class, y_pred),
        "f1": f1_score(y_test_class, y_pred, average='weighted'),
        "recall": recall_score(y_test_class, y_pred, average='
           weighted'),

    })

    # Store predictions in dictionary
    comparison_dict[name] = y_pred

# Convert to DataFrames at the end
results_df = pd.DataFrame(results).sort_values(by="f1", ascending
   =False)
comparaison = pd.DataFrame(comparison_dict)


display(results_df)
display(comparaison)
```

```
model                      accuracy     f1         recall
-----------------------------------------------------------
RandomForest               0.774238   0.767743   0.774238
DecisionTreeClassifier     0.702216   0.702068   0.702216
SVC                        0.620499   0.589463   0.620499
GaussianNB                 0.574792   0.572336   0.574792

real_values  RandomForest  DecisionTreeClassifier  SVC  Gaussian
-----------------------------------------------------------------
0            0             1                       0    0
0            0             0                       1    0
0            1             0                       1    1
0            0             0                       0    0
0            0             0                       0    1
...
1            0             1                       0    0
0            0             0                       1    1
1            0             0                       0    1
0            0             0                       0    0
0            0             0                       0    0
```

The results indicate that the Random Forest classifier outperforms other models across all evaluation metrics. Its superior performance highlights the advantage of ensemble methods in capturing complex interactions between features.

Decision Trees achieve moderate performance, while SVC and Gaussian Naive Bayes show lower predictive capability, suggesting limitations in handling the dataset's structure and feature dependencies.

```python
df_melted = results_df.melt(id_vars="model", var_name="Metric",
    value_name="Score")

# 2. Plotting
plt.figure(figsize=(12, 6))
sns.barplot(x="Score", y="model", hue="Metric", data=df_melted,
    palette="viridis")

# 3. Aesthetics
plt.title("Model Performance Comparison", fontsize=15)
plt.xlabel("Score (0.0 - 1.0)", fontsize=12)
plt.ylabel("Model", fontsize=12)
plt.xlim(0, 1)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left') # Moves
    legend outside the plot
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()

plt.show()
```

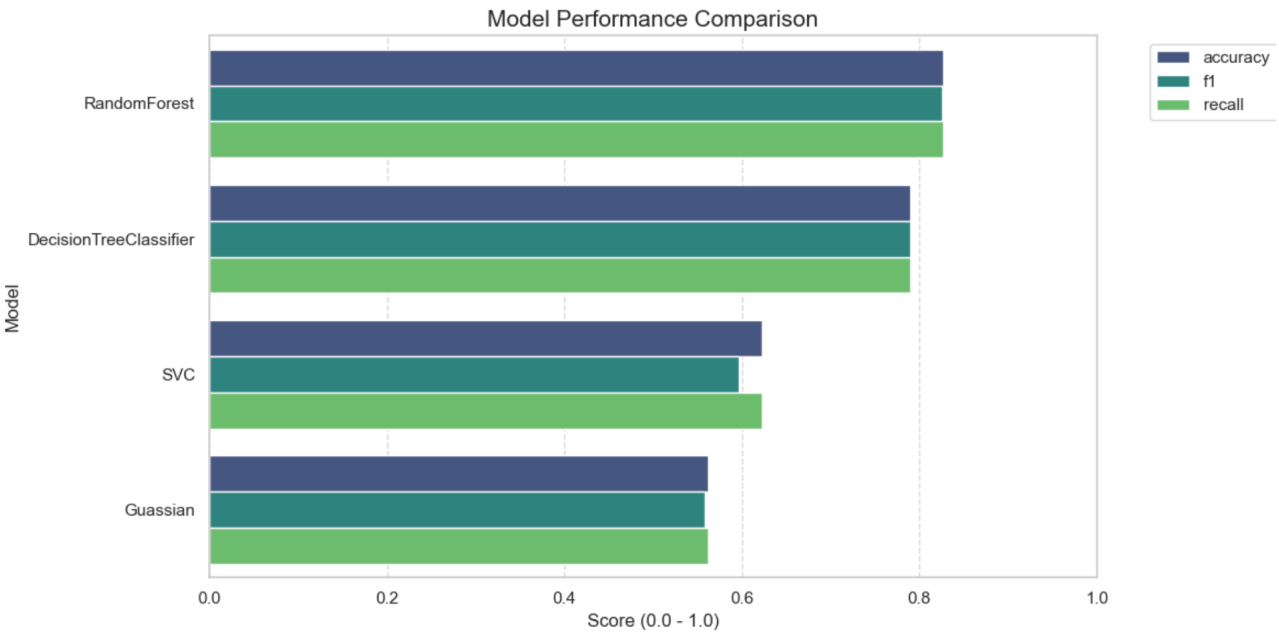


Figure 4.2: Performance Comparison of Binary Classification Models

Figure 4.2 compares model performance across accuracy, F1-score, and recall metrics. The visualization confirms the dominance of the Random Forest model, which consistently achieves higher scores, indicating balanced and reliable classification performance.

4.2.4 Confusion Matrix

```
for name, model in models.items():

    model.fit(X_train, y_train_class)
    y_pred = model.predict(X_test)
    fig, ax = plt.subplots(figsize=(10, 8))
    cm = confusion_matrix(y_test_class, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
       display_labels=["Class 0", "Class 1"])
   disp.plot(ax=ax, cmap='Blues', values_format='d')
 ax.set_title(f"Detailed Confusion Matrix: {name}", fontsize=18,
    fontweight='bold')
    ax.set_xlabel('Predicted Label', fontsize=14)
    ax.set_ylabel('True Label', fontsize=14)

    plt.show()
```

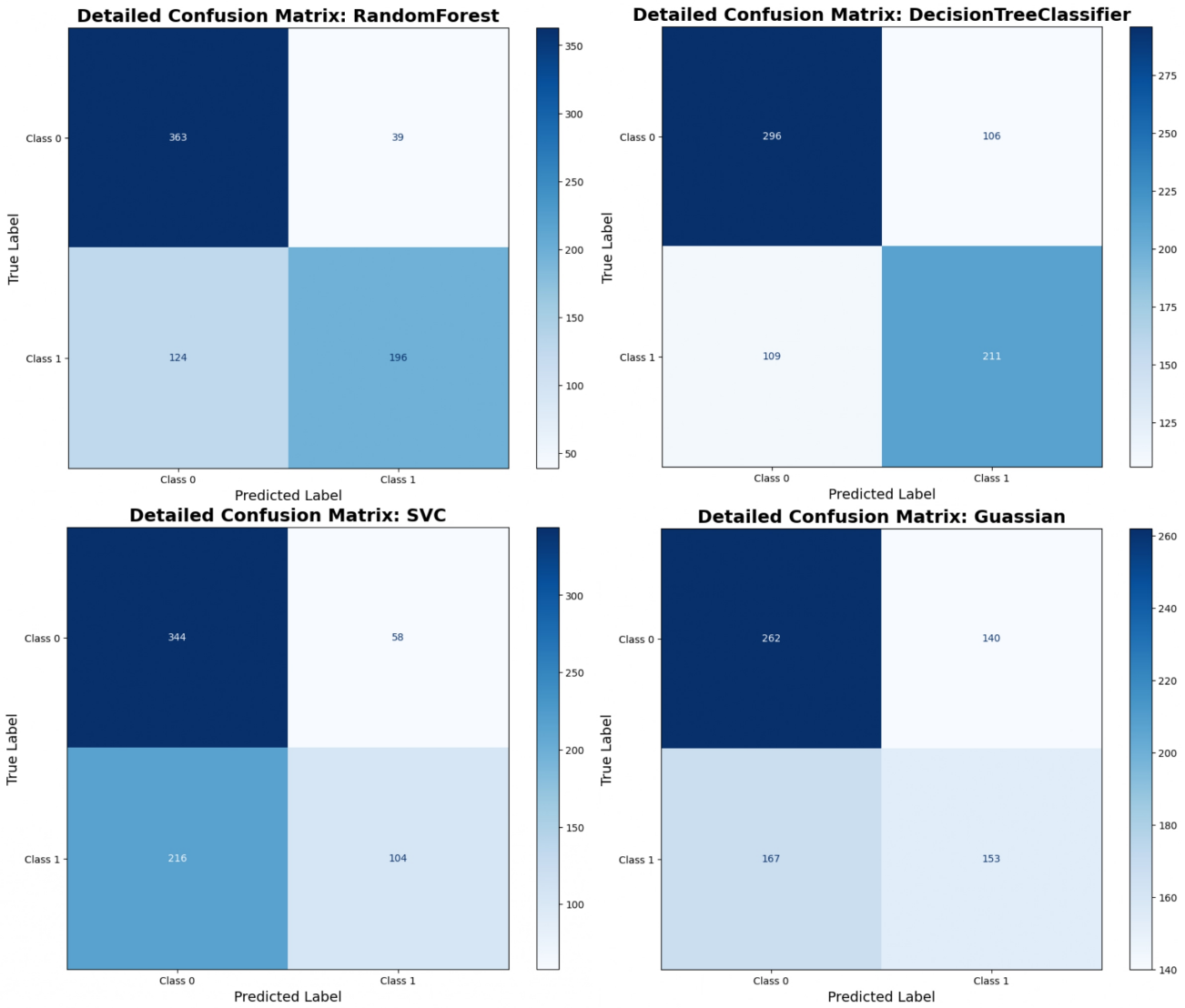


Figure 4.3: Confusion Matrices of Binary Classification Models

The confusion matrices in Figure 4.3 provide a detailed view of each model's classification behavior.

The Random Forest classifier demonstrates the best balance between true positives and true negatives, while other models exhibit higher misclassification rates. These results reinforce the selection of Random Forest as the most effective model for binary academic impact prediction in this study.

Chapter 5

Clustering

5.1 Feature Selection

In this step, all encoded and scaled features are selected to perform the clustering analysis. Using the fully preprocessed dataset ensures that all variables are numerical and comparable, which is a crucial requirement for distance-based algorithms such as K-Means. This approach guarantees that no single feature dominates the clustering process due to differences in scale or data type, allowing each variable to contribute fairly to the formation of clusters.

```
X_clustering = data_encoded
```

5.2 The Optimal Number of Clusters (Elbow Method)

To determine the optimal number of clusters, the Elbow Method is applied. This method evaluates the Within-Cluster Sum of Squares (WCSS) for different values of k in order to measure cluster compactness.

```
wcss = []
K_range = range(1, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state
       =42)
    kmeans.fit(X_clustering)
    wcss.append(kmeans.inertia_)

# Affichage du graphique du coude
plt.figure(figsize=(10, 5))
plt.plot(K_range, wcss, marker='o', linestyle='--', color='b')
plt.title('Elbow Method to determine optimal K', fontsize=14)
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.grid(True)
plt.show()
```

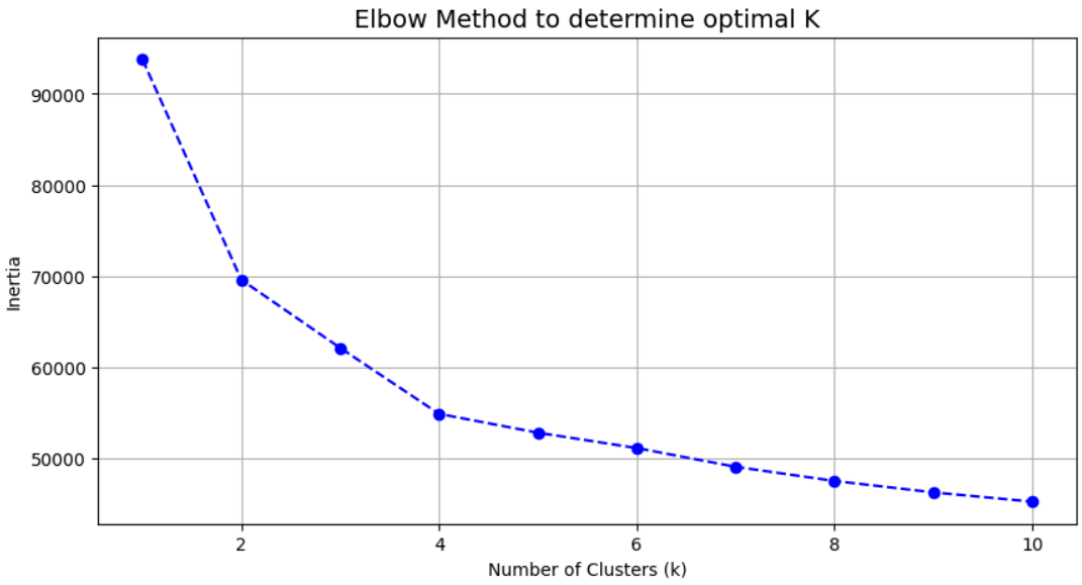


Figure 5.1: Elbow Method for Determining the Optimal Number of Clusters

Figure 5.1 illustrates the evolution of the clustering inertia as the number of clusters increases. The visible elbow in the curve indicates the most suitable number of clusters.

5.3 Training the K-Means Model

Based on the elbow method, four clusters were selected as a reasonable trade-off between model simplicity and representational capacity.
The K-Means algorithm is then trained to group students into distinct segments based on similarities in their encoded features.

```
# Optimal number of clusters selected based on the elbow method
optimal_k = 4
kmeans = KMeans(n_clusters=optimal_k, init='k-means++',
    random_state=42)

# Assigning each student to a cluster
data['Cluster'] = kmeans.fit_predict(X_clustering)
data_encoded['Cluster']=data['Cluster']
print(f"Success! {optimal_k} student segments have been created."
    )
data[['Daily_Usage_Hours', 'Cluster']].head()
```

Table 5.1: Clustering Based on Daily Usage Hours

ID	Daily Usage Hours	Cluster
0	0.9	3
1	3.4	1
2	3.6	2
3	2.9	0
4	0.9	3

Table 5.1 provides an initial overview of cluster assignment with respect to daily AI usage hours. The distribution shows clear variability between clusters, confirming that usage intensity is a key dimension in the clustering process.

5.4 Visualizing Clusters in 2D (PCA)

To visualize the clustering results, Principal Component Analysis (PCA) is applied to reduce the high-dimensional feature space into two principal components.

```
# Dimensionality Reduction (4D -> 2D)
pca = PCA(n_components=2)
pca_data = pca.fit_transform(X_clustering)

# Creating the DataFrame for visualization
pca_df = pd.DataFrame(data=pca_data, columns=['PC1', 'PC2'])
pca_df['Cluster'] = data['Cluster']

# Plot
plt.figure(figsize=(12, 8))
sns.scatterplot(x='PC1', y='PC2', hue='Cluster', data=pca_df,
    palette='viridis', s=100, alpha=0.7)
plt.title('Student Segmentation Visualization (PCA)', fontsize
    =16)
plt.legend(title='Cluster Group')
plt.show()
```

The following scatter plot shows the distribution of students across the different clusters in a two-dimensional space, highlighting the separation between groups.

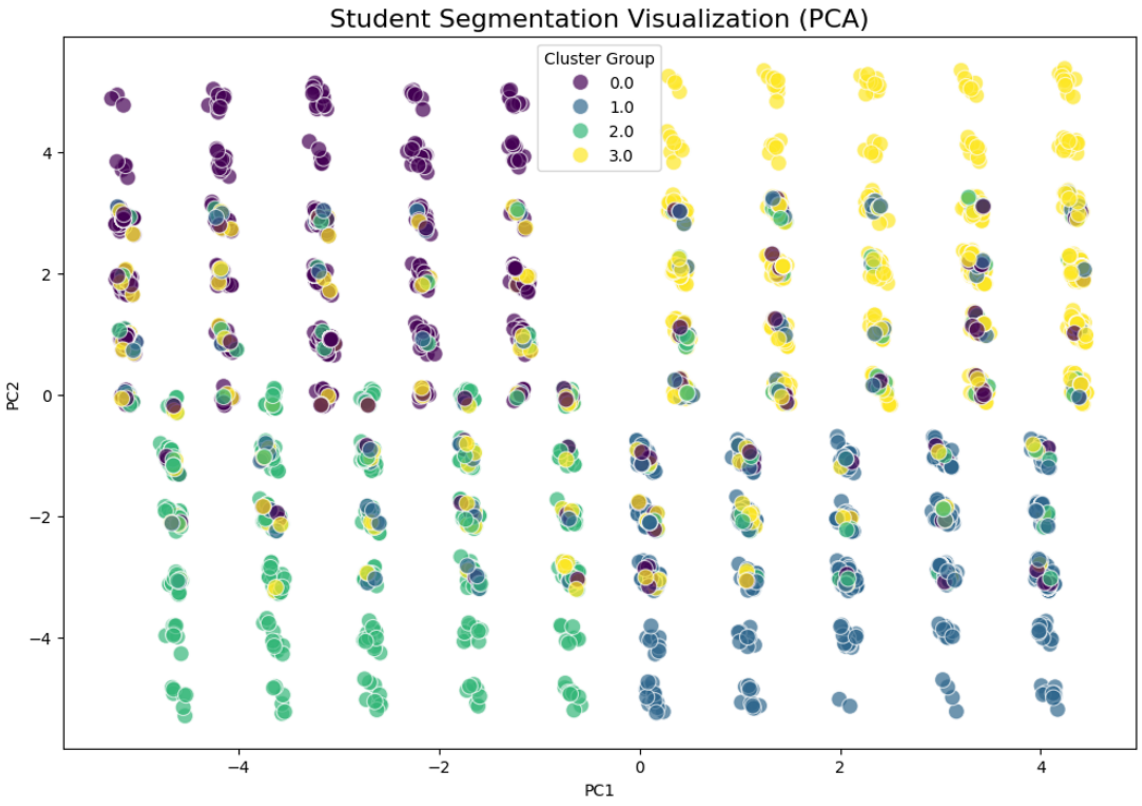


Figure 5.2: Two-Dimensional Visualization of Student Clusters Using PCA

5.5 Cluster Profiling (Analysis)

Cluster profiling is performed by computing the mean value of each feature within every cluster. This analysis helps identify the key characteristics that differentiate student groups.

```
# 1. Sync the Cluster labels to your encoded DataFrame
data_encoded['Cluster'] = kmeans.labels_

# 2. Group by Cluster and calculate the mean
cluster_analysis = data_encoded.groupby('Cluster').mean(
    numeric_only=True)

plt.rcParams.update({'font.size': 20})

fig, ax = plt.subplots(figsize=(22, 35), dpi=100)


cluster_analysis.T.plot(kind='barh', ax=ax, width=0.8, colormap='
    viridis')

plt.title('DETAILED CLUSTER ANALYSIS (Horizontal View)', fontsize
    =40, fontweight='bold', pad=40)
plt.xlabel('Mean Scaled Value', fontsize=30, labelpad=20)
plt.ylabel('Features', fontsize=30, labelpad=20)

ax.legend(title='Clusters', loc='upper center', bbox_to_anchor
    =(0.5, -0.05),
          ncol=3, fontsize=25, title_fontsize=30, frameon=True,
              shadow=True)

plt.axvline(0, color='black', linewidth=2)
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.subplots_adjust(bottom=0.1)
plt.show()
```

Figure 5.3 provides a detailed comparison of the main features across clusters, allowing a clear interpretation of behavioral and usage differences.

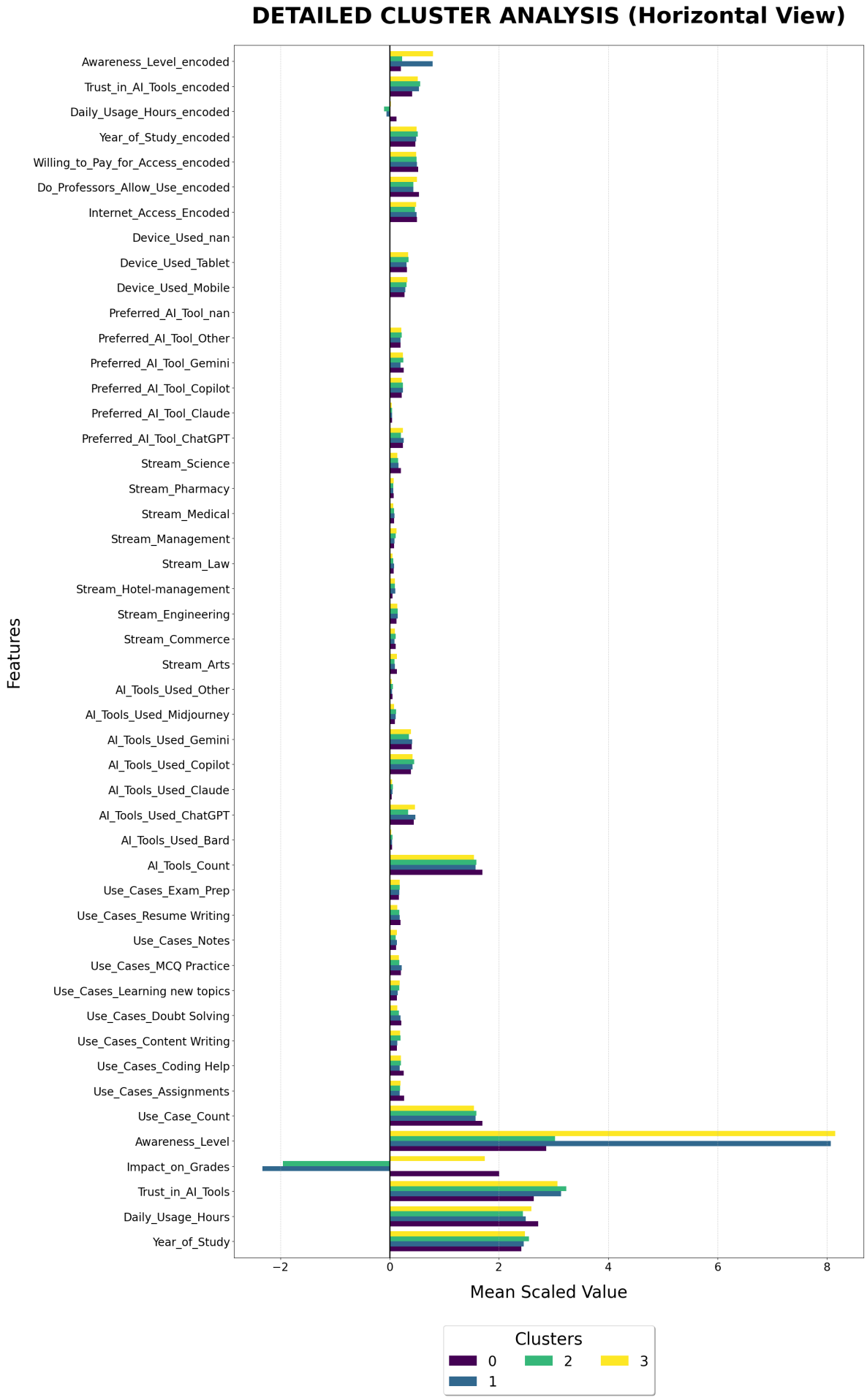


Figure 5.3: Cluster Profiling Based on Mean Encoded Feature Values

Chapter 6

Conclusion

6.1 Project Workflow Summary

This project followed a complete data mining workflow, starting from data collection and exploratory analysis to predictive modeling and clustering.

The process began with an in-depth exploratory data analysis to understand students' AI usage patterns and identify relevant relationships between variables. Data cleaning and preprocessing were then performed to ensure quality and consistency, including encoding categorical variables, scaling numerical features, and selecting meaningful attributes.

Supervised learning techniques were applied to predict the academic impact of AI usage, followed by unsupervised clustering to segment students into distinct behavioral groups. Each step was carefully designed to maintain statistical validity and interpretability.

6.2 Key Findings

The analysis revealed that AI tools are widely used by college students, primarily for academic support tasks such as coding assistance, assignments, and exam preparation. Correlation and statistical tests showed that the impact of AI usage on grades is influenced by multiple factors, including daily usage duration and professors' permission to use AI tools. While these relationships are statistically significant, their strength remains relatively weak, indicating that academic performance is shaped by a combination of factors beyond AI usage alone.

Classification results demonstrated that binary prediction of academic impact yields better performance than multi-class classification, with the Random Forest model achieving the highest accuracy and F1-score. Clustering analysis further highlighted the heterogeneity of student behavior, revealing distinct groups with varying levels of AI engagement, awareness, and academic impact.

Although the impact of AI tools on academic performance appears to be moderate, this study confirms that it is statistically significant. The findings indicate that the effect of AI usage on grades depends more on usage patterns and educational context than on usage intensity alone, emphasizing the importance of responsible and pedagogically guided use of AI tools in higher education.

6.3 Study Limitations and Contributions

Despite the valuable insights obtained, this study presents certain limitations. The dataset is based on self-reported survey responses, which may introduce subjective bias in variables such as perceived academic impact. Additionally, the observed correlations remain weak, limiting the predictive power of the models.

Nevertheless, the study makes a significant contribution by providing a comprehensive data-driven analysis of AI tool usage in higher education. It demonstrates the practical application of data mining techniques to real-world educational data and offers an interpretable framework for understanding students' interactions with AI technologies.

6.4 Future Work and Perspectives

Future work could focus on incorporating additional academic performance indicators, such as objective grade records, to strengthen predictive accuracy.

Exploring advanced models and longitudinal data could also provide deeper insights into how AI usage behavior evolves over time and its long-term academic implications.

Finally, extending the analysis to different educational contexts or countries would allow broader generalization of the findings and support more informed policy decisions regarding AI integration in education.